

© Akshat Paul and Abhishek Nalwaya 2019

Akshat Paul and Abhishek Nalwaya, *React Native for Mobile Development*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-4454-8_9

9. Getting Ready for the World

Akshat Paul¹ and Abhishek Nalwaya²

(1) Gurgaon, Haryana, India

(2) Jaipur, Rajasthan, India

The last 10 percent to launch something takes as much energy as the first 90 percent.

—Rob Kalin

That's a strong quote, but it usually proves quite right. However, when it comes to launching our application with React Native, it is a piece of cake. This chapter covers how you can create a build to distribute your React Native application for both iOS and Android. We also introduce some third-party platforms that are available to make distribution totally stress free. Here are the topics covered in this chapter:

- The Apple and Google Play distribution systems
- Creating a build for iOS or Android
- Beta testing

Apple Developer Account

To create builds for iOS devices for testing or to distribute applications to actual users on the Apple App Store, you must first create and pay for an Apple Developer Account. An individual developer account costs around US\$99 and works perfectly for independent developers, or even for developers who are part of a small organization. However, if your company policy requires you to be a member of a team, then your company can go open Enterprise account. Table [9-1](#) lists the differences so you can determine what works best for your organization.

Table 9-1. Selecting Your Options

	Only Apple ID	Apple Developer Program	Enterprise Program
Xcode developer tools	✓	✓	✓
Xcode beta releases	✓	✓	✓
Test on device	✓	✓	✓
Developer forums	✓	✓	✓
OS beta releases	×	✓	✓
Advanced app capabilities	×	✓	✓
Code-level support	×	✓	✓
Distribution outside Apple App Store	×	✓	✓

	Only Apple ID	Apple Developer Program	Enterprise Program
App Store distribution	×	✓	×
App Store connect	×	✓	×
Safari extensions	×	✓	×
Offering custom apps	×	✓	×
Distribution of custom apps to your employees	×	✓	×
Distribution of your proprietary apps to your employees	×	×	✓
Cost	Free	US\$99 (annually)	US\$299 (annually)

To create your Apple Developer Account, visit

<https://developer.apple.com/>.

Google Play Console

In the case of Android, you explicitly do not need to have a paid account from Google Play at the time of development or testing. However, eventually

when you have to distribute your application (i.e., publish it to the Google Play Store) you would have to pay a one-time registration fee of US\$25. However, this will only be required toward the end of the chapter if you wish to publish your app to the Play Store. To learn more about the Google Play Console visit <https://play.google.com/apps/publish/>.

iOS Build Process

To get ready, we must set up our Apple Developer Profile. Apple has a specific way of setting up certificates, IDs, and profiles. Not to worry; we'll learn about them all by setting them up for our React Native app. Once you have your paid Developer account, log in into <https://developer.apple.com/>. You'll have two options, as shown in Figure 9-1.

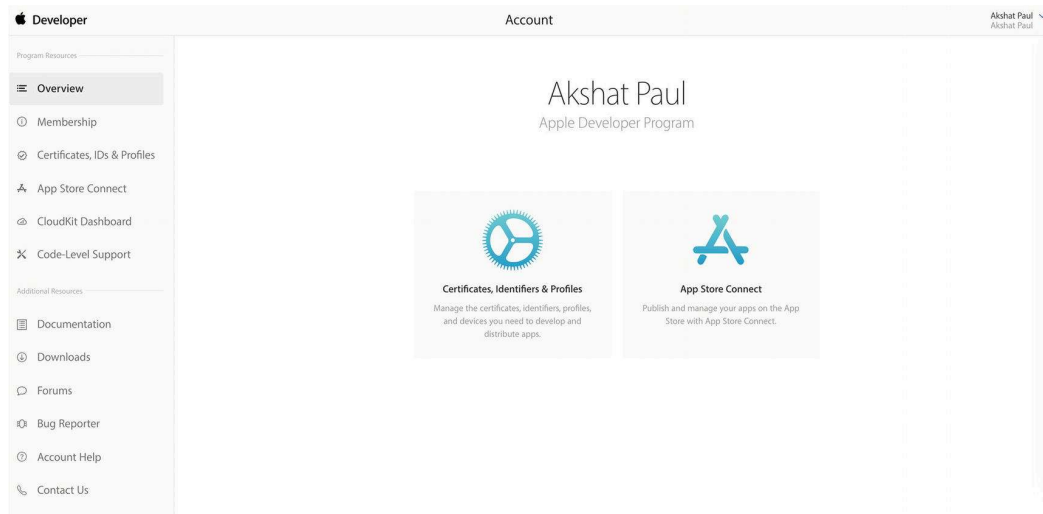


Figure 9-1. Apple Developer console

The first option is Certificates, Identifiers & Profiles and the second one is App Store Connect. The App Store is the place where we will upload our ap-

plication build to be submitted to Apple for publishing our app to the App Store and also for beta testing our application using TestFlight.

Open the Certificates, Identifiers & Profiles page, select Development on the left, and then click the plus (+) button on the right to begin the development and distribution certificate process (Figure [9-2](#)).

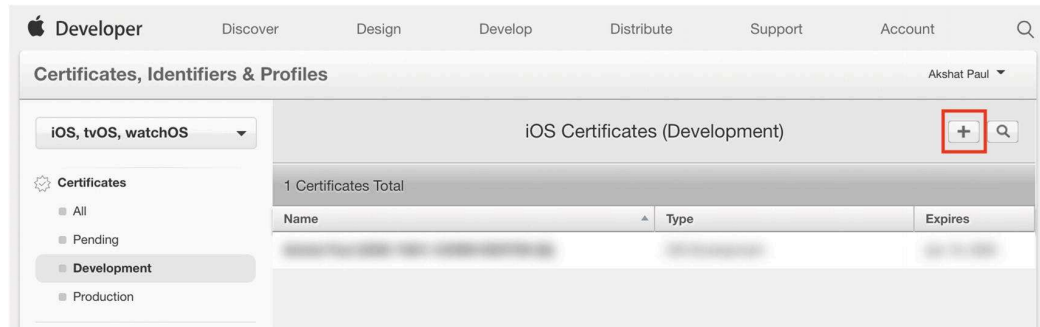


Figure 9-2. Beginning the development and distribution process

You will then have the option to create either a development or distribution certificate (Figure [9-3](#)).

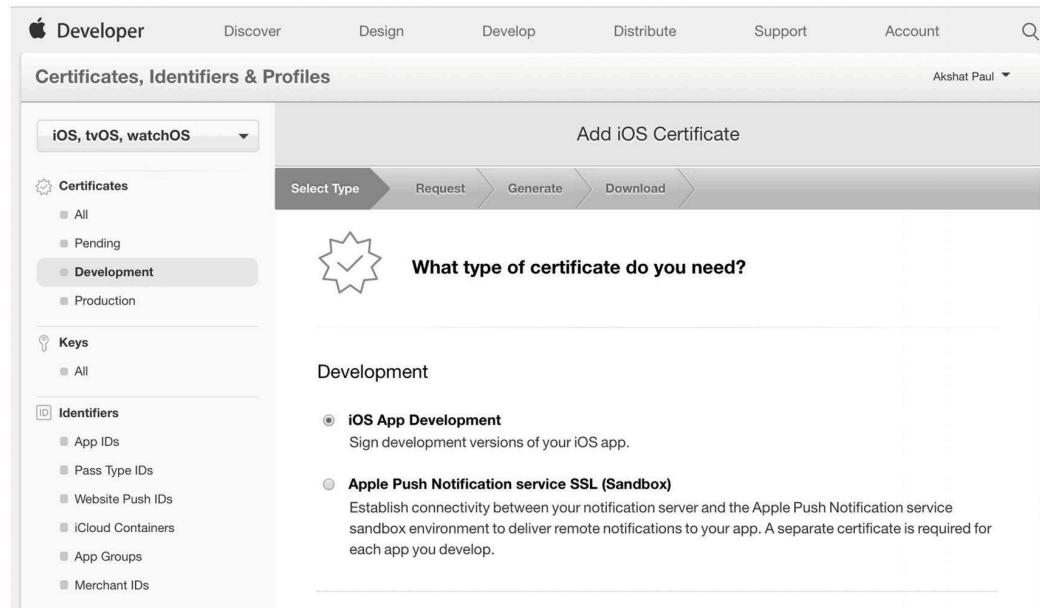


Figure 9-3. Selecting development and distribution certificates

We will require both because we plan to publish our application to the App Store and the process is the same for both. Select iOS App Development and continue to the next step. There you will see instructions how to generate the certificate on your Mac machine, which will be then uploaded. For this you will make use of the Keychain utility.

Create a CSR File

In the `Applications` folder on your Mac, open the `Utilities` folder and launch Keychain Access. From the Keychain Access drop-down menu, select Keychain Access ► Certificate Assistant ► Request a Certificate from a Certificate Authority. In the Certificate Information window, enter the following information:

1. In the User Email Address field, enter your e-mail address.
2. In the Common Name field, create a name for your private key (e.g., John Doe Dev Key).
3. The CA Email Address field should be left empty.

4. In the Request is group, select Saved to disk.

Click Continue within Keychain Access to complete the CSR generating process. Once this is complete, click Continue and you see an option to upload the CSR file in your developer portal (Figure 9-4).

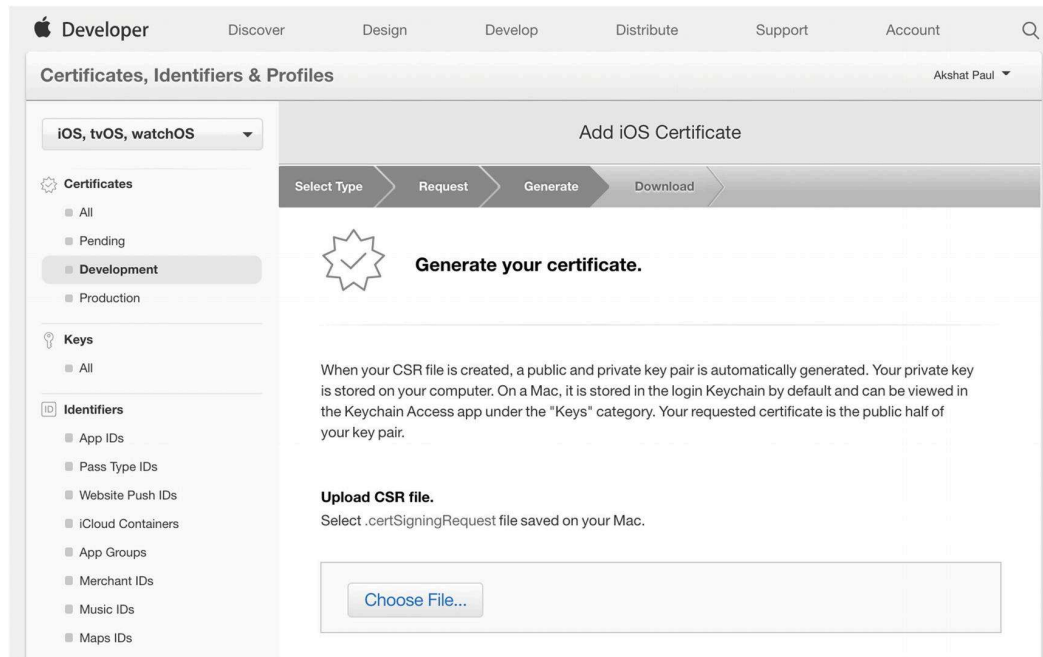


Figure 9-4. Certificate upload page on Apple Developer portal

Once the CSR file is uploaded, click Continue. At the last step you will have the option to download the certificate. Double-click it and it will get loaded in your Keychain.

Next, follow the same steps and set up distribution certification. On successful completion you can check both your installed certificates in Keychain ► My Certifications.

We next create an App ID that will be unique for every application. Under Identifiers, select App IDs and then click the plus (+) button (Figure 9-5).

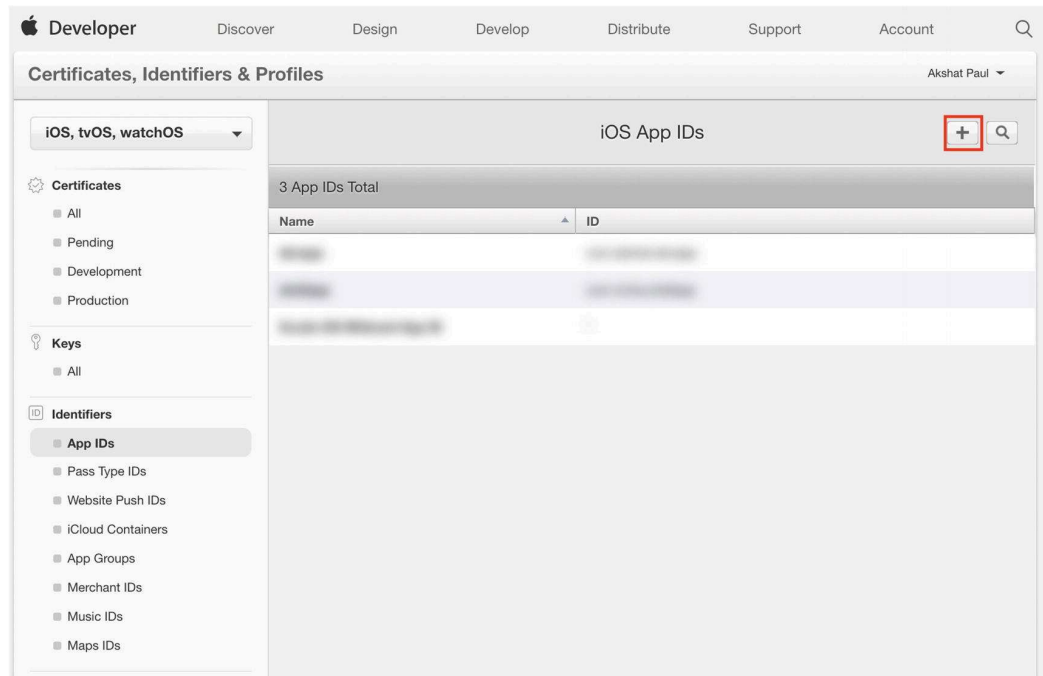


Figure 9-5. List of App ID page on Developer portal

That will open the screen shown in Figure 9-6.

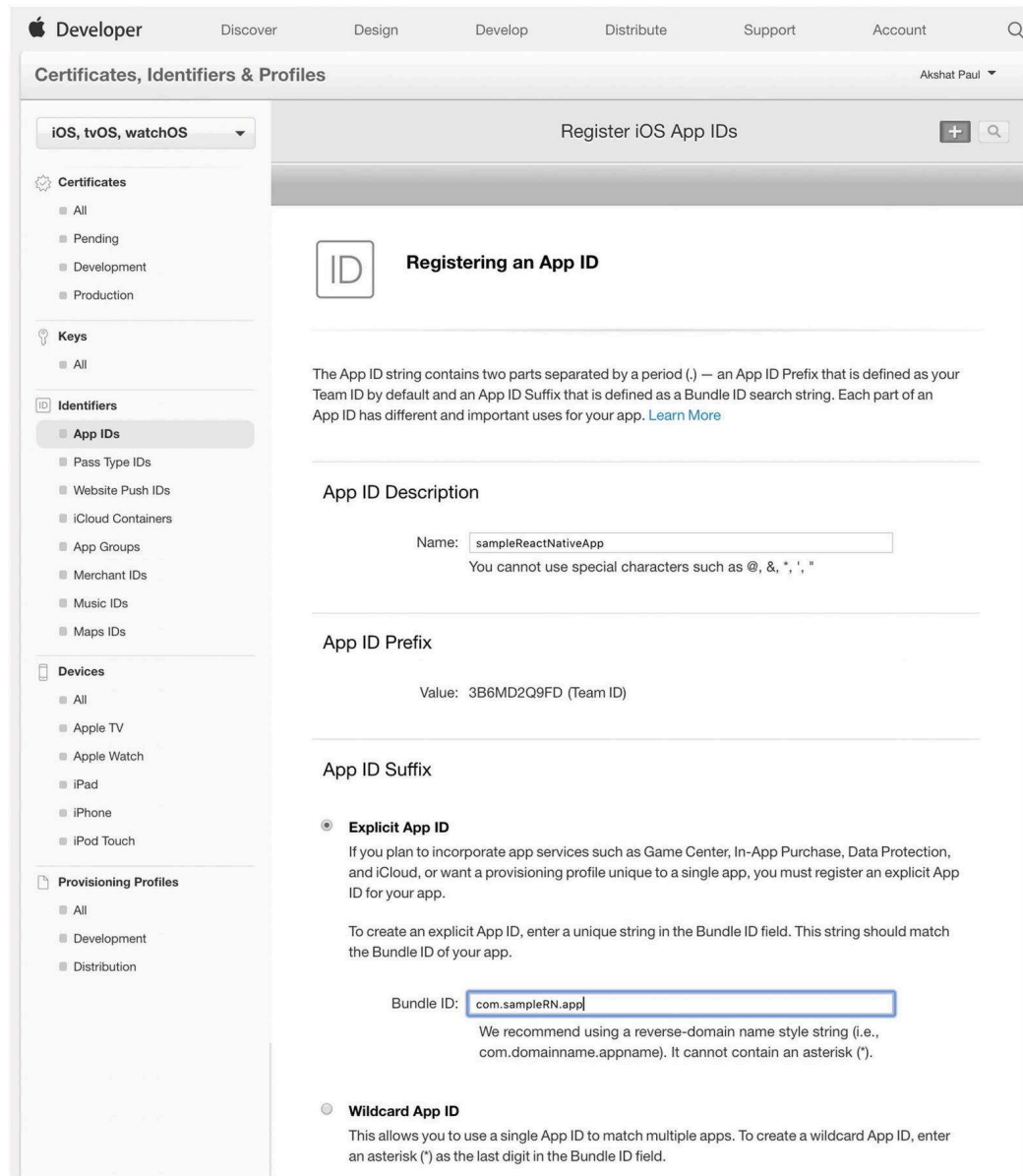


Figure 9-6. Registering an App ID in the Apple Developer portal

Make a note of the Bundle ID, as it's the same ID that we have to use in our App ID. We will use this once we open our code to create the build for our application. In our example we have named it `com.sampleRN.app`, but you can use any nomenclature you desire. Click Continue, and your App ID will be listed within the App IDs section (Figure 9-7).

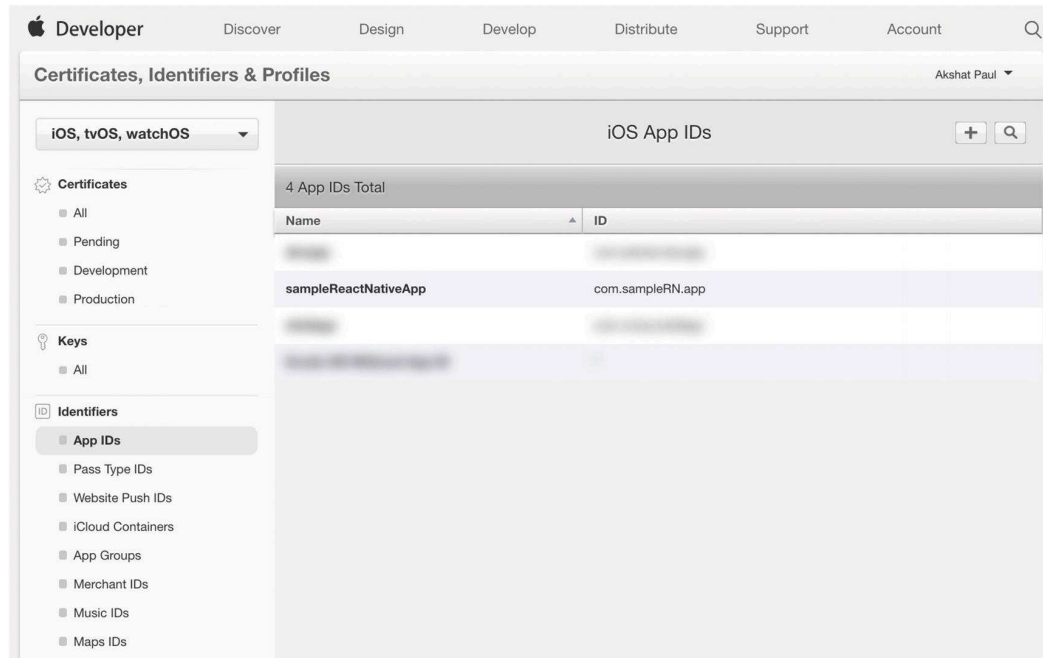


Figure 9-7. App ID shown in the Apple Developer portal

Next, we create a Development and Distribution profile for the `sampleReactNative` application. Scroll down to the Provisioning Profiles section and select Development, then click the plus (+) button (Figure [9-8](#)).

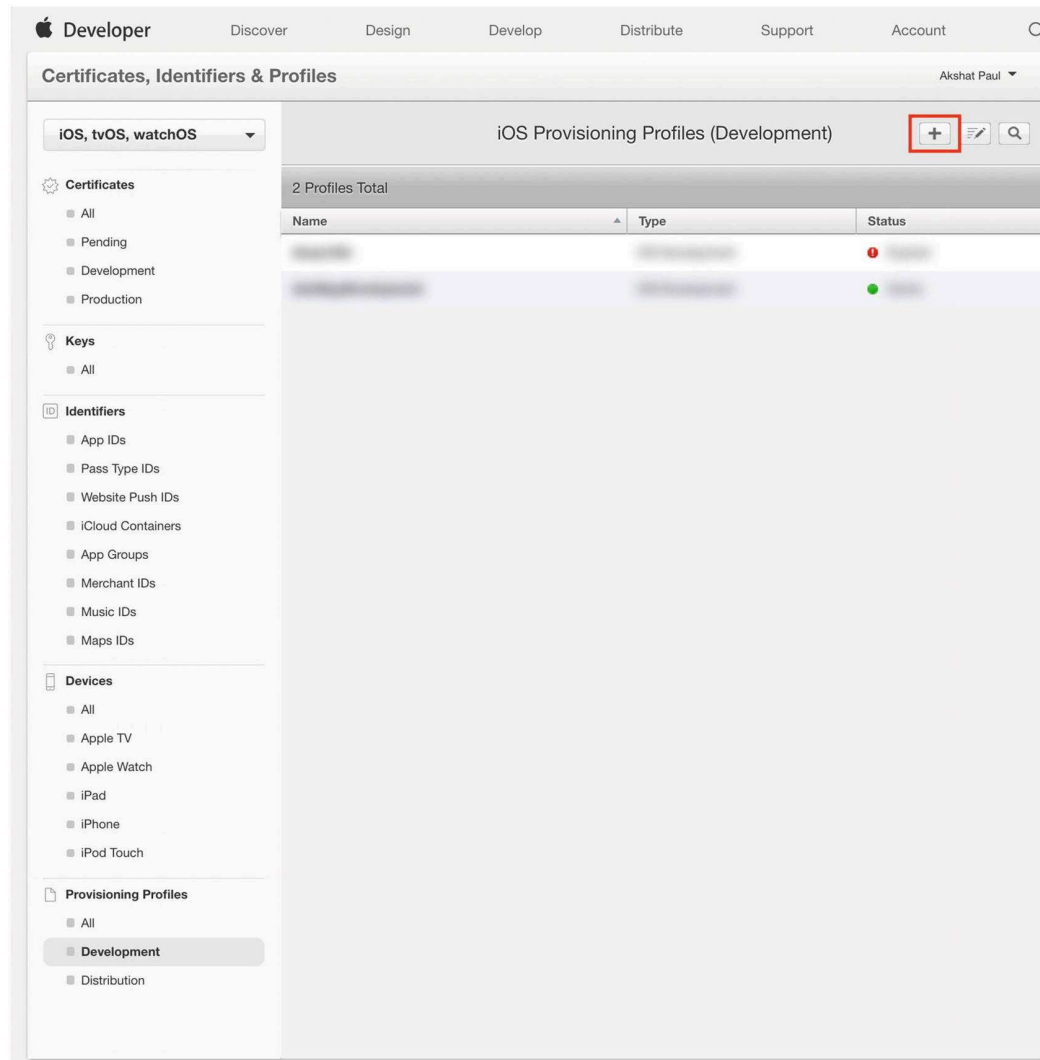


Figure 9-8. iOS Provisioning Profiles list page in the Apple Developer portal

Select the appropriate App ID from the drop-down list. In our case it will be the App ID we created in the previous section for our `sampleReactNative` app (Figure [9-9](#)).

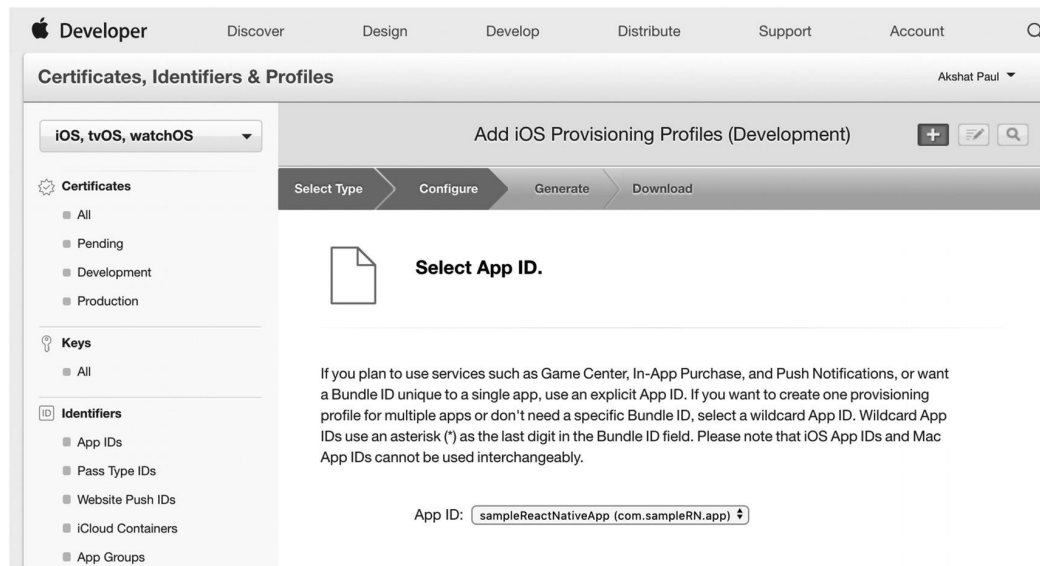


Figure 9-9. Add iOS Provisioning Profiles page on the Apple Developer portal

Click Continue. Your Development Provisioning profile will be generated. Double-click it and it will be loaded in your Xcode automatically. Before proceeding to the next section, create a Distribution Provisioning profile for our `sampleReactNative` app using the same process.

Now that all our basic setup is completed, in the next section you learn how we create a build for our application and distribute it among our team members using TestFlight.

Generating iOS .ipa(iPhone Application Archive)

Before we create our build and host it on TestFlight for testing, we should load our source code in Xcode. From the root of your React Native source code, navigate to the appropriate folder and click the Xcode project file (Figure [9-10](#)).

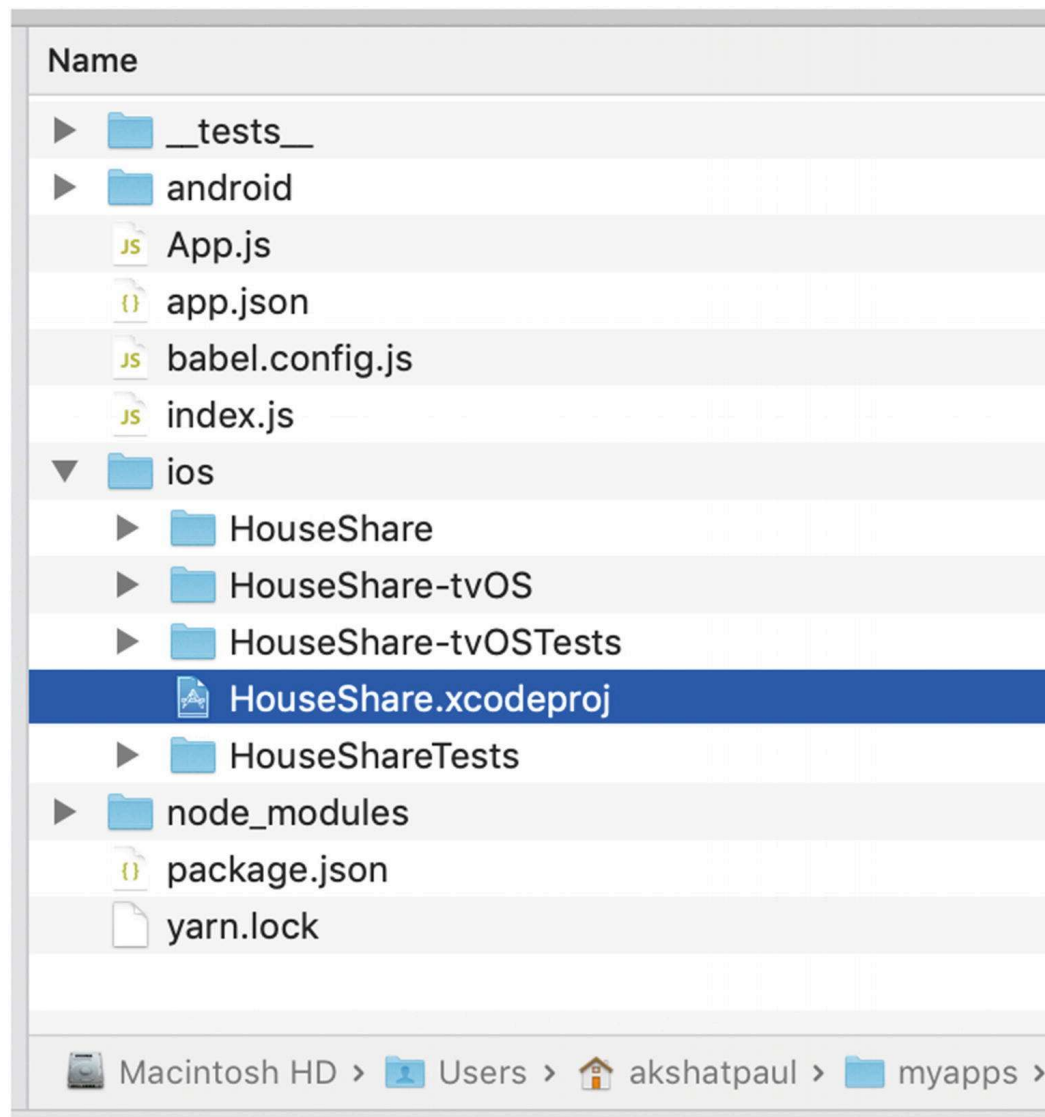


Figure 9-10. Folder structure of iOS project

Double-click this to load your application in Xcode. Click the General tab for the application to add the settings shown in Figure [9-11](#).

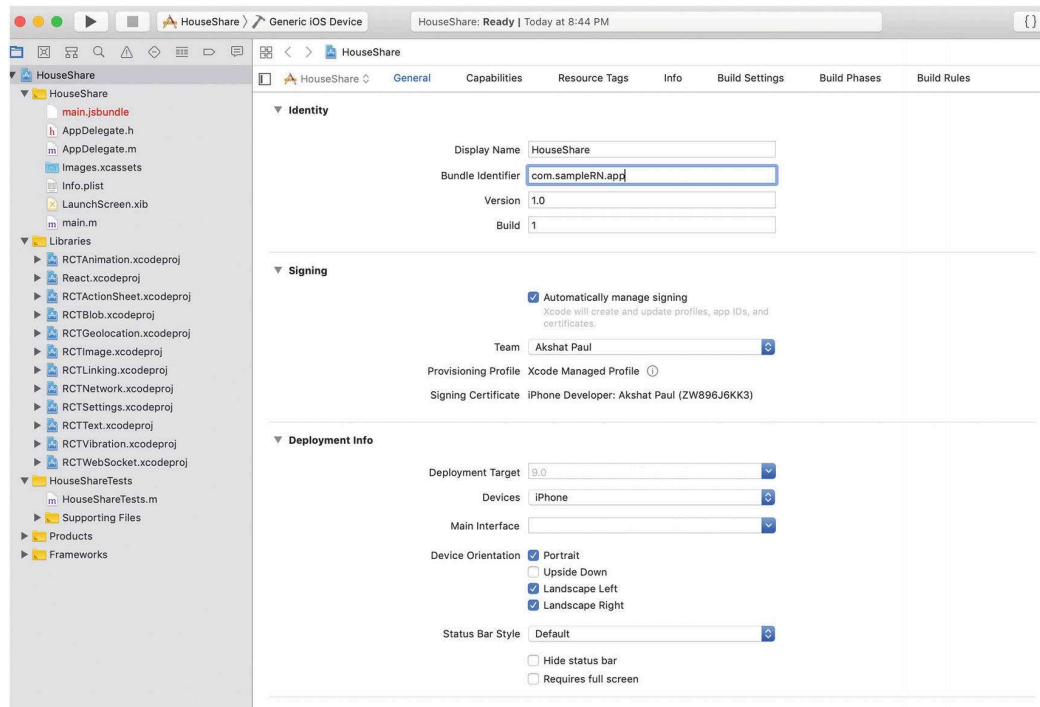


Figure 9-11. Xcode General tab settings for the project

Use the App ID you had created in previous section as the Bundle Identifier. This has to be same, as mentioned in the Developer console, and unique for every application you create. It is essentially a unique identifier for your app in Apple's system.

Next, let's add some app icons and a launch screen for our sample application. It's fine to keep your launch screen simple, with just text that comes out of the box when you initialize a React Native application. However, we must add all types of icons for our build to be successful and submitted to Apple for both App Store release and testing with TestFlight.

To add icons to your application, select Images.xcassets ► AppIcon folder from the project directory from Xcode (Figure [9-12](#)).

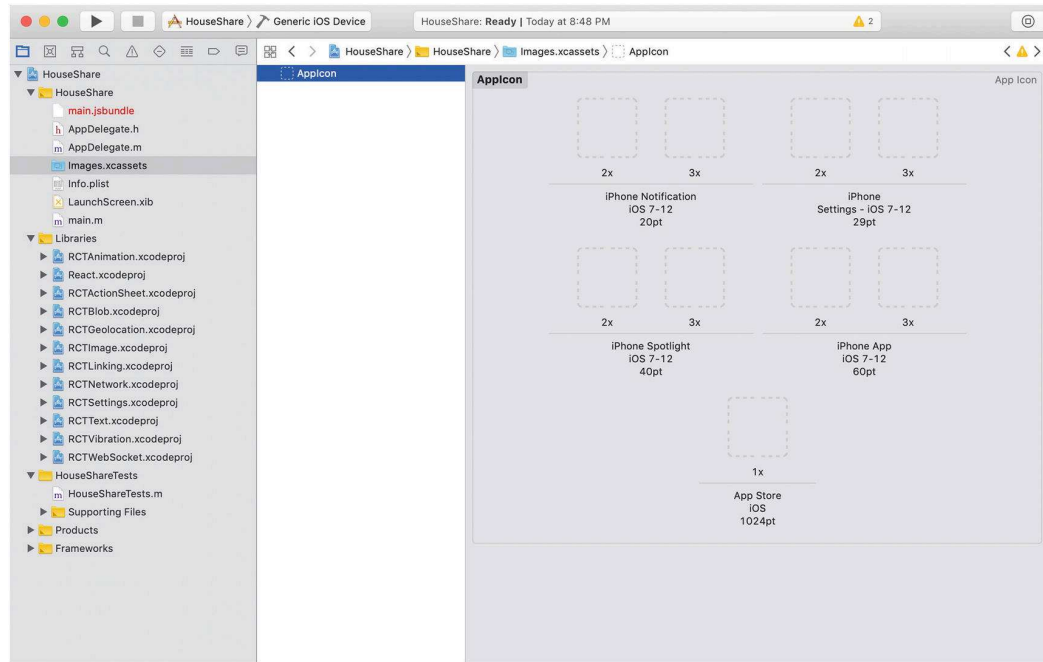


Figure 9-12. Icon image set screen in Xcode

The icons shown here represent the same icon for your application to be used at different places; in short, they represent your application icon in various sizes. We won't get into a tutorial here about how to create these icons, because that's a designer's area of expertise. For our work, we can use some application—we suggest downloading Icon Set Creator for your Macintosh—to generate all sizes of icons for iOS devices (Figure [9-13](#)). There are many online sites that can help you perform the same task.

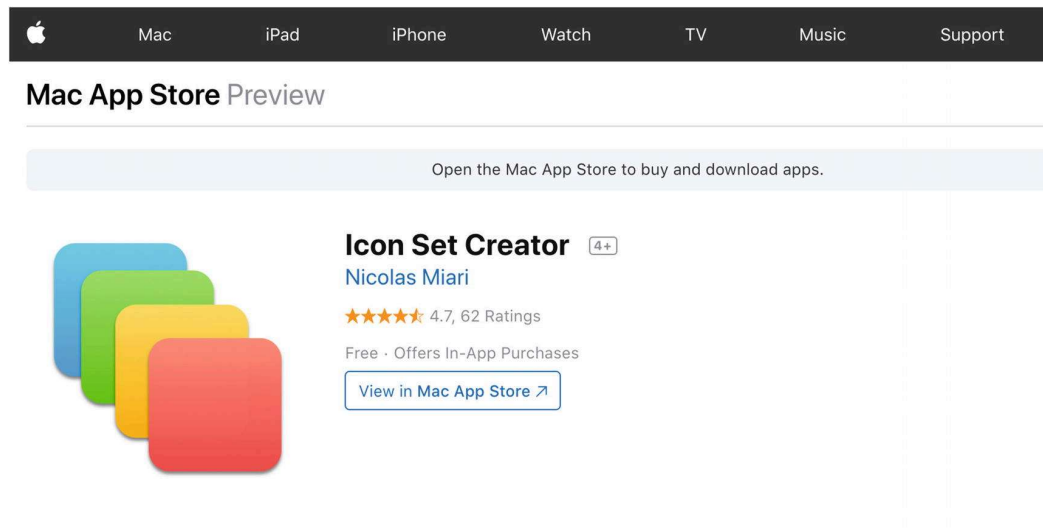


Figure 9-13. An app for creating an icon set on Mac

Next drag and drop your icons into the ApplIcon pane, as shown in Figure [9-14](#).

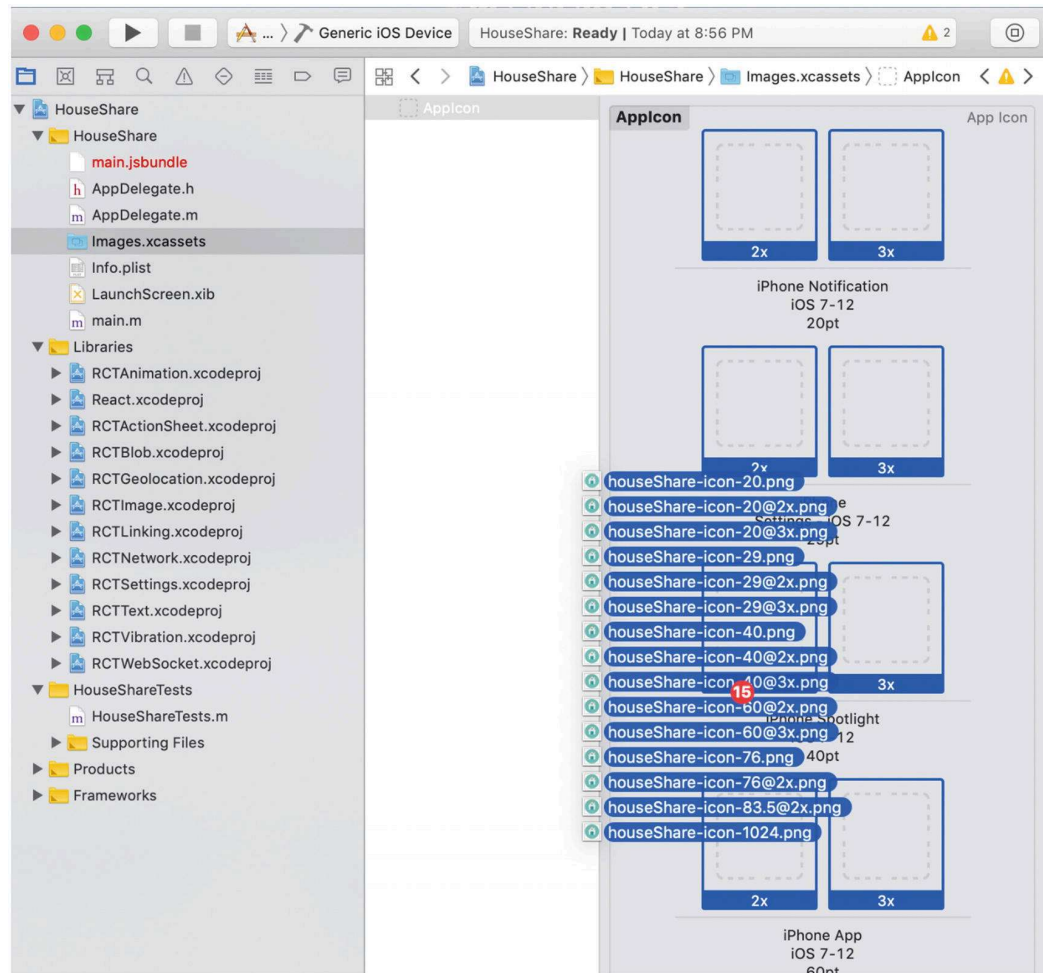


Figure 9-14. Icon image set screen in Xcode

Once this is done, you will see all your icons automatically set up and you're ready for the next step, which is setting up your launch screen.

Select `LaunchScreen.xib` to add or modify the launch screen for your application. In our sample application, we will keep the same default launch screen because it won't break our build or stop us from uploading it. However, for a real-world application that is supposed to be published to the App Store, it is better to have a proper launch screen.

Next, let's create our build, which is actually done using the Archive command. Before we create the build, please select Generic iOS Device as the target, as shown in Figure 9-15. The reason for changing this from a simulator to Generic iOS Device is that your Archive command will be disabled if you don't make this change. On the Xcode menu bar, select Product ➤ Archive and the build process will begin.

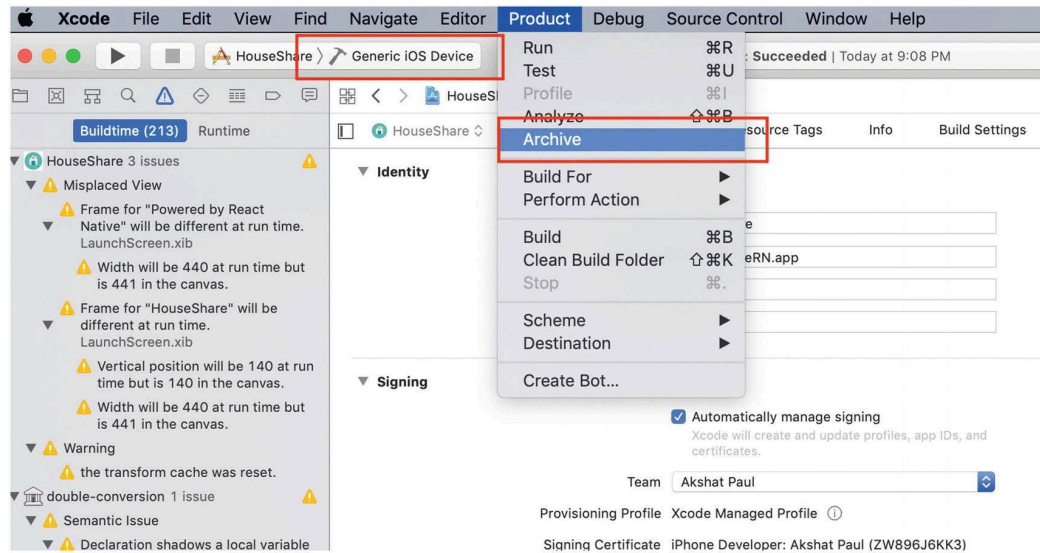


Figure 9-15. Xcode archive generation

When successfully built, you'll get the window shown in Figure 9-16 with a list of all your Archives. If this window does not appear for some reason, even after a successful build or by mistake you close it, you can reopen it. This is Xcode Organizer.

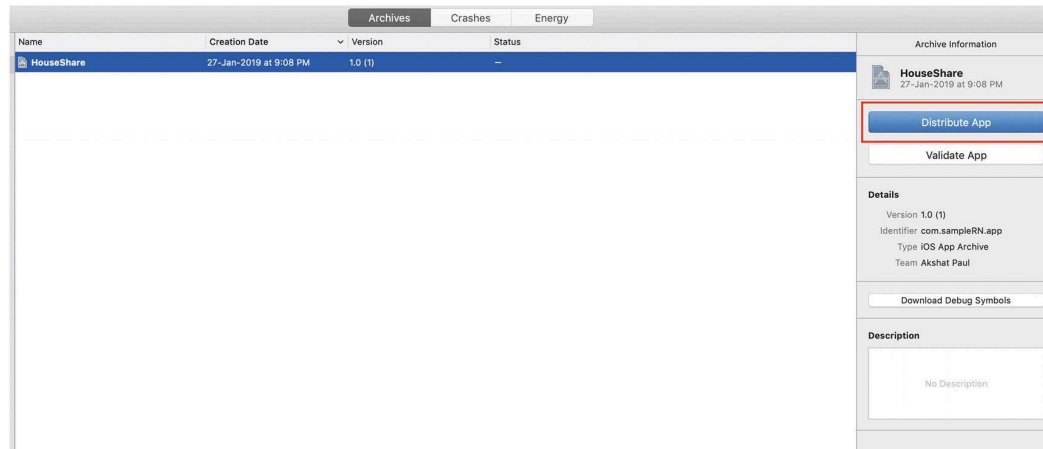


Figure 9-16. Xcode Organizer

Click Distribute App and you will be presented with a few options. Select iOS App Store and after few steps your ipa will be ready to be uploaded to App Store Connect (Figure [9-17](#)).

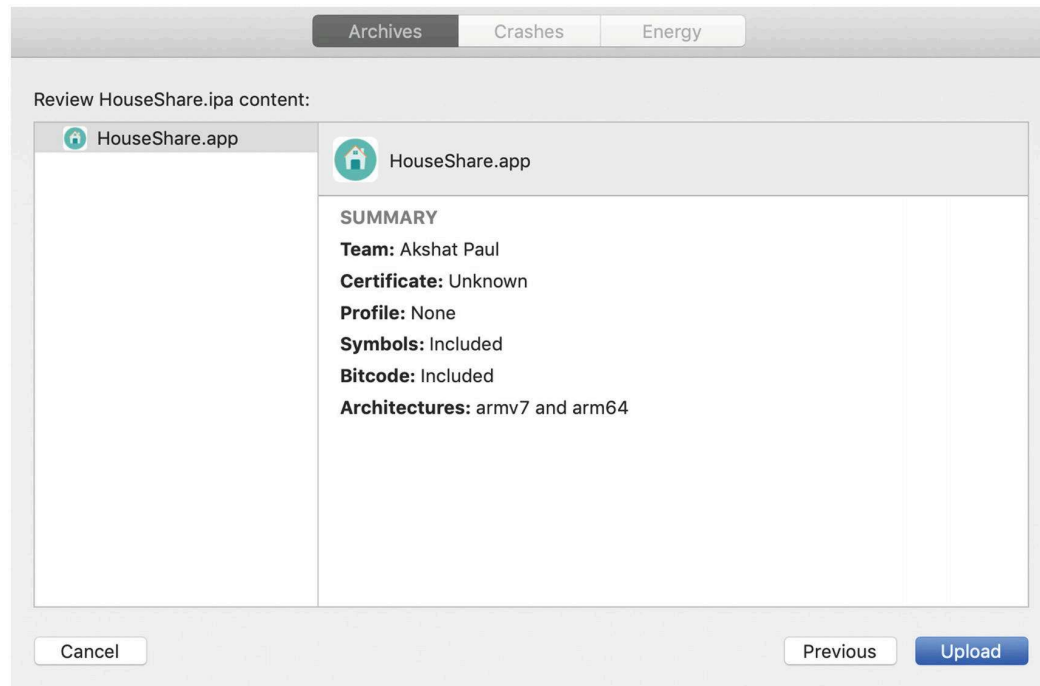


Figure 9-17. Xcode Organizer detail screen

Before clicking Upload, you need to first create the application on the App Store Connect. Go to <https://developer.apple.com/account> and select the App Store Connect icon or visit <https://appstoreconnect.apple.com/>. There you'll find several options. Select My Apps, as shown in Figure 9-18.

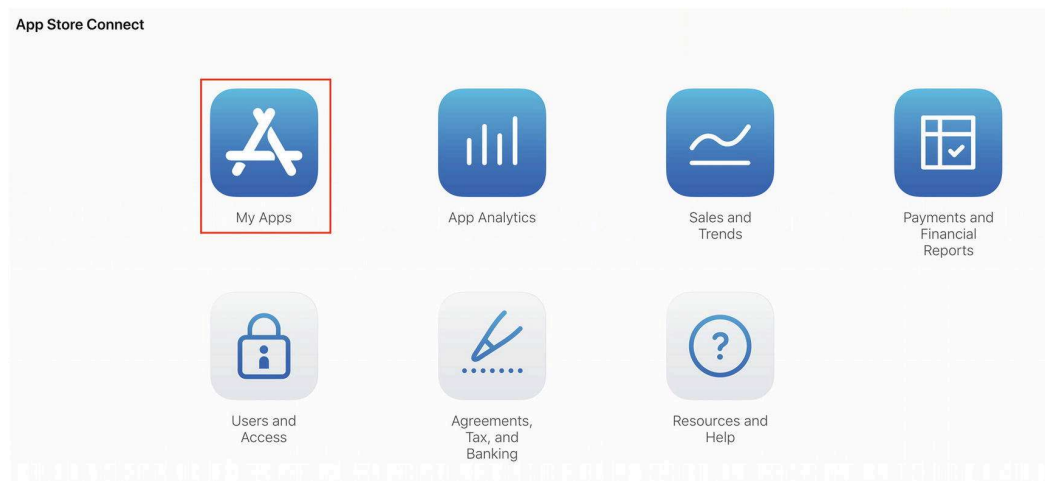


Figure 9-18. App Store Connect home page

Inside My Apps you will see all your iOS applications. Click the plus (+) button and select New App to create new App Store app for our React Native application (Figure 9-19).

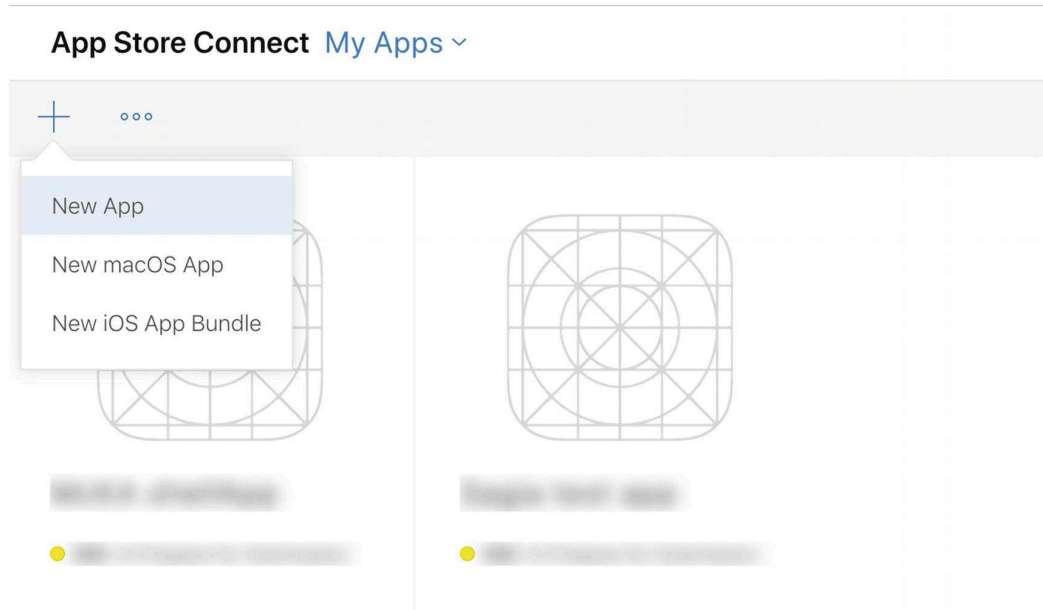


Figure 9-19. App Store Connect create new app

Once selected, the form displayed in Figure [9-20](#) will appear.

New App

Platforms ?

☒ iOS ☐ tvOS

Name ?

House Share

Primary Language ?

English (Australia) | v

Bundle ID ?

sampleReactNativeApp - com.sampleRN.app | v

SKU ?

come.sampleRN2019.app

User Access ?

☐ Limited Access ☒ Full Access

Cancel Create

Figure 9-20. App Store Connect form to create a new app

Fill it out with the proper details for your application. You can select the appropriate Bundle ID from the drop-down list. A SKU has to be added, which can be different from the Bundle ID. This SKU is not visible to App Store users. For user access, if you have created any specific user group already you can select it. If not, select Full Access, especially if this is your first application.

Click Create and your empty app will be created on App Store Connect. Go back to Xcode and continue where we left off. Click Upload and shortly your application build will be uploaded on App Store Connect (Figure 9-21).

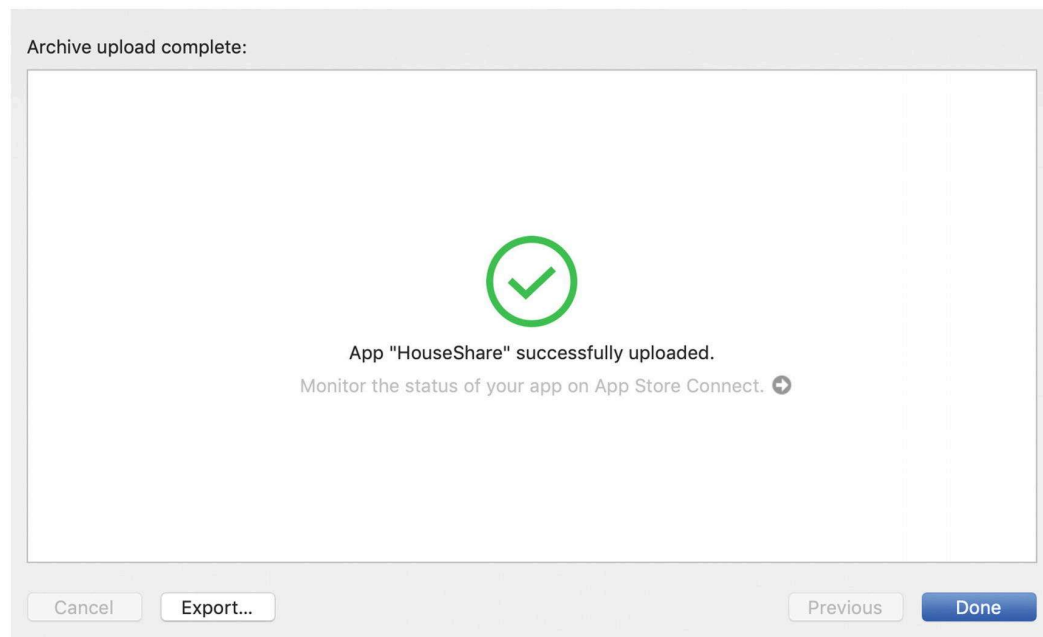


Figure 9-21. Organizer success screen

You can check your application build on App Store Connect in a few. From App Store Connect, you can submit your application to Apple for review. After a successful review of your application without issues and errors, your app will be live on the Apple App Store for users in two to five days.

Before you publish your application for end users, it must be thoroughly tested. This process is called beta testing and can be achieved using TestFlight.

Generating Android .apk(Android application package)

Just like Apple, Google expects all Android apps to be signed with a certificate before they get installed on a device either for testing or publishing in the Google Play Store.

To begin this process, first navigate to the folder where your Java Development Kit (JDK) is installed. In the case of Macintosh, if you are not sure where your JDK is installed, type the following command in your terminal:

```
$ /usr/libexec/java_home
```

This will print the path to the `jdk` folder. Navigate to that directory and type the following command:

```
$ sudo keytool -genkey -v -keystore my-release-key.keystore -alias my-key-alias -keyalg RSA -keysize 2048 -validity 10000
```

When you execute this command, it will ask few questions and require a password to be set for your keys. Please remember the password because it will be used later when applying these settings for your React Native application.

Copy the `my-release-key.keystore` file in the `android/app` directory in your React Native application folder (Figure [9-22](#)).

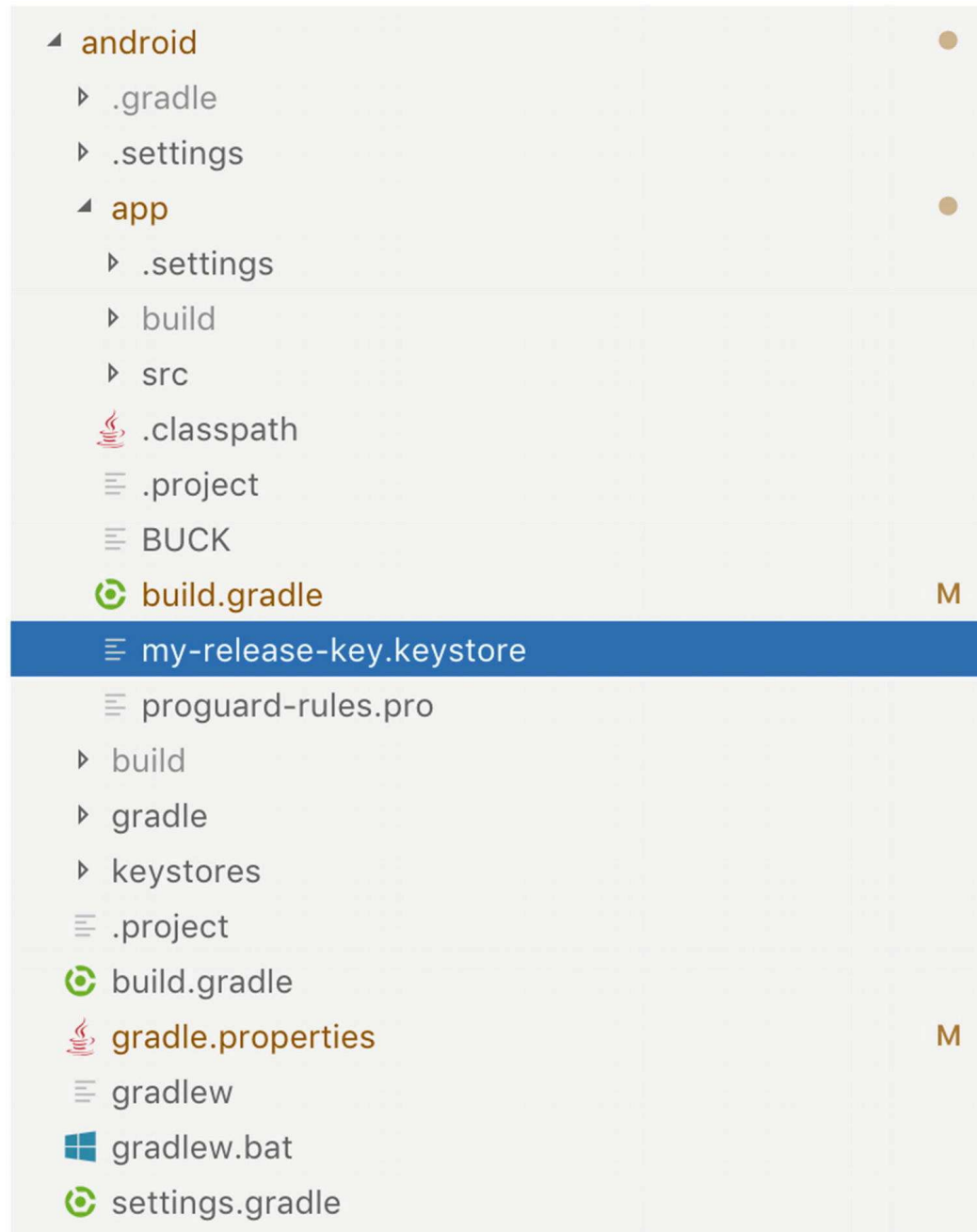


Figure 9-22 Android folder structure of app

Note Always make sure to keep your keys private and never commit in the project directory.

Next, we need to add some configurations in two files. First open `android/gradle.properties` and add the following settings:

```
MYAPP_RELEASE_STORE_FILE=my-release-key.keystore
MYAPP_RELEASE_KEY_ALIAS=my-key-alias
MYAPP_RELEASE_STORE_PASSWORD=*****
MYAPP_RELEASE_KEY_PASSWORD=*****
```

As mentioned earlier, provide the password you set when you were generating your keys.

Next, open `android/app/build.gradle`. In `signingConfigs` ➤ `release` section, add the following config:

```
signingConfigs {
    release {
        if (project.hasProperty('MYAPP_RELEASE_STORE_FILE')) {
            storeFile file(MYAPP_RELEASE_STORE_FILE)
            storePassword MYAPP_RELEASE_STORE_PASSWORD
            keyAlias MYAPP_RELEASE_KEY_ALIAS
            keyPassword MYAPP_RELEASE_KEY_PASSWORD
        }
    }
}
```

In the same file inside the `buildTypes` ➤ `release` section, add the following config:

```
buildTypes {
```

```
    release {  
      ...  
      signingConfig signingConfigs.release  
    }  
  }  
}
```

Finally, to generate an apk, go to the `android` folder in your React Native application and execute the following command:

```
$ gradlew assembleRelease
```

This will generate the apk build that you can find at `android/app/build/outputs/apk/release/app-release.apk`. This apk can be distributed to users and submitted to the Google Play Store.

Beta Testing with TestFlight

TestFlight is a utility that is included when you set up your Apple Developer Account. It allows you to invite users to test your application, provide you with feedback, and provide you with valuable test information like crashes, and so on.

Each build is active for 90 days and you can invite up to 25 internal testers (which does not require App Store review) and up to 10,000 external testers, which is only applicable after App Store review.

Let's also set up our sample React Native application for TestFlight. The process is pretty simple. Inside App Store Connect, select your application and click the TestFlight tab (Figure [9-23](#)).

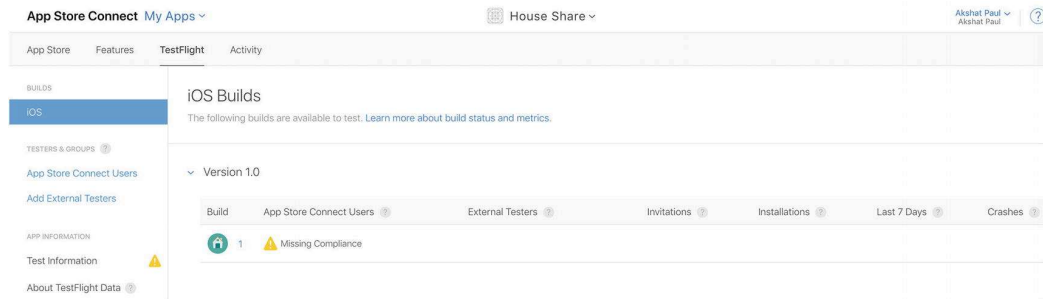


Figure 9-23 App Store Connect TestFlight tab

You will see the recently uploaded build available. It will mention missing compliance. Under App Information, select Test Information from the menu pane. Click the Missing Compliance message again and click Start Internal Testing.

You can invite up to 25 users to participate in internal testing. To add users, return to the App Store Connect home screen and select Users and Access. From there, you can add your testing users and segregate them into groups if required.

Your testers would have to install the TestFlight application from the Apple App Store to access the build, which will be installed separately on your iOS device.

TestFlight is a good option, but it is limited to only iOS device testing. Besides TestFlight we would recommend TestFairy and HockeyApp as alternatives that can be used for both iOS and Android. Whereas TestFairy is a paid utility, HockeyApp is completely free (at the time of this writing).

Summary

In this chapter we finally reached the end of the development cycle for a mobile application, creating a build that can be tested by users and submitted to the Apple App Store or the Google Play Store. You learned about the signing process for both systems: Whereas Apple has specific steps in its signing process, the Google Android process is fairly quick. Both, though, are designed to keep the rights and devices of users from being misused. You also learned about beta testing with TestFlight and some other popular options.